Computer Vision

DPS920

Session 23

Overview

Deep Learning
Hyperparameter Tuning
Learning Rate
Overfitting
Batch Normalization
Data Augmentation
Adam, ReLu

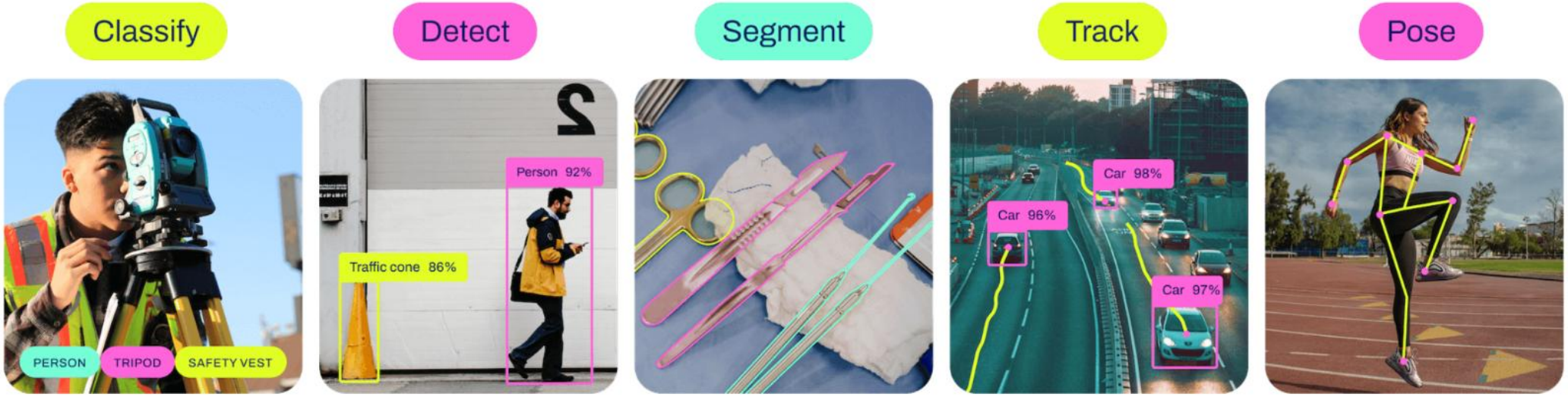
Agenda

YOLO
Ultralytics
Object Detection
Segmentation
Pose Estimation
Tracking
Medical Use cases

What is Left?

1. Optimization and Loss Function
2. Code + Logistic Regression
3. ML and Images
4. Perceptron and Neural Networks
5. Neural Networks
6. Convolution Neural Networks (CNN)
7. Advanced CNNs
8. Introduction to object detection, Segmentation and image generation methods with AI

Some Vision Tasks



Traditional:

- Viola-Jones (Haar Cascades),
- HOG + SVM (Histogram of Oriented Gradients + Support Vector Machine)
- Selective Search + SVM
- EdgeBoxes

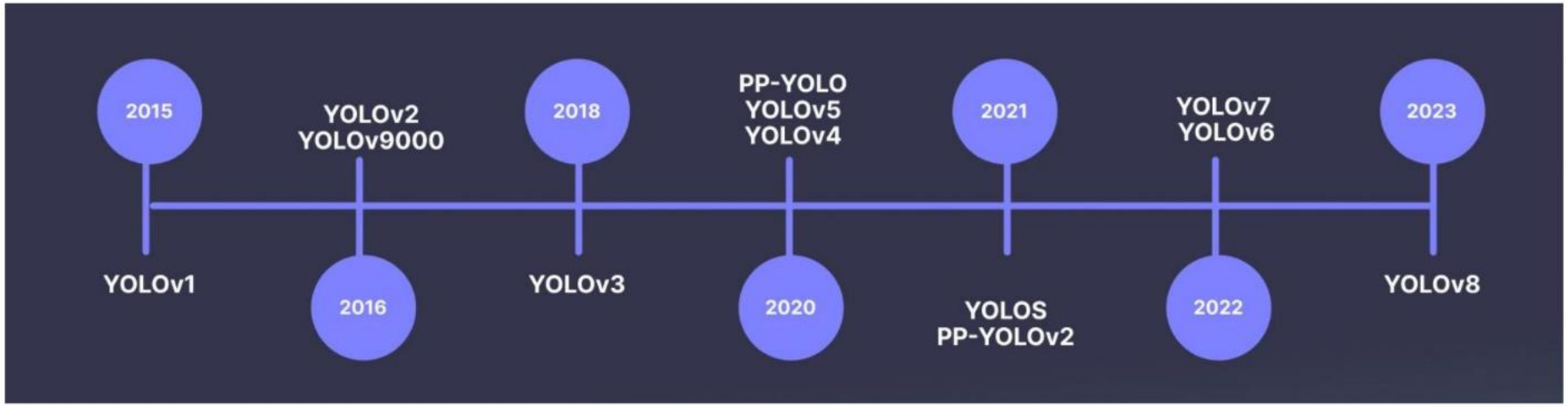
Deep Learning 2 stage:

R-CNN
Fast R-CNN
Faster R-CNN
Mask R-CNN (adds segmentation)
Cascade R-CNN
Libra R-CNN
DCN (Deformable Convolutional Networks)

Deep Learning 1 stage:

- YOLO (You Only Look Once)
- SSD (Single Shot MultiBox Detector)
- RetinaNet
- EfficientDet
- CenterNet
- FCOS (Fully Convolutional One-Stage Object Detection)
- CornerNet
- YOLO-NAS (Neural Architecture Search)

YOLO

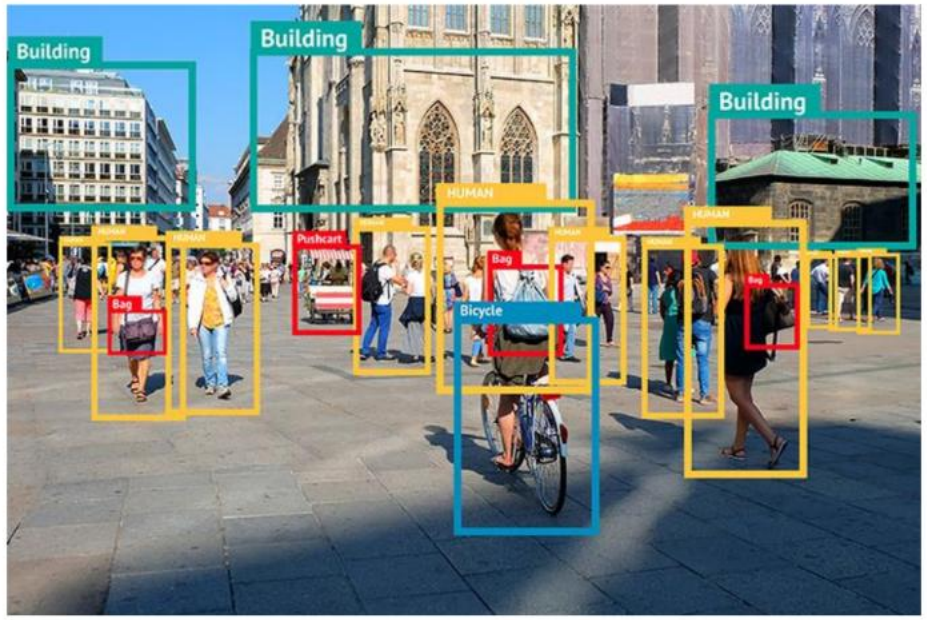


Joseph Redmon

Ultralytics

- pip install ultralytics

Object Detection



object, name, confidence score

YOLOv8

YOLOv8n	YOLOv8s	YOLOv8m	YOLOv8l	YOLOv8x
3.2M	11.2M	25.9M	43.7M	68.2

Example

```python
import cv2
from ultralytics import YOLO

# Load a pretrained YOLOv8n model
model = YOLO('yolov8n.pt')

# Read an image using OpenCV
source = cv2.imread('path/to/image.jpg')

# Run inference on the source
results = model(source)  # list of Results objects
```

```python
from ultralytics import YOLO

# Load a pretrained YOLOv8n model
model = YOLO('yolov8n.pt')

# Define remote image or video URL
source = 'https://ultralytics.com/images/bus.jpg'

# Run inference on the source
results = model(source)  # list of Results objects
```

```python
import numpy as np
from ultralytics import YOLO

# Load a pretrained YOLOv8n model
model = YOLO('yolov8n.pt')

# Create a random numpy array of HWC shape (640, 640, 3) with values in range
source = np.random.randint(low=0, high=255, size=(640, 640, 3), dtype='uint8')

# Run inference on the source
results = model(source)  # list of Results objects
```

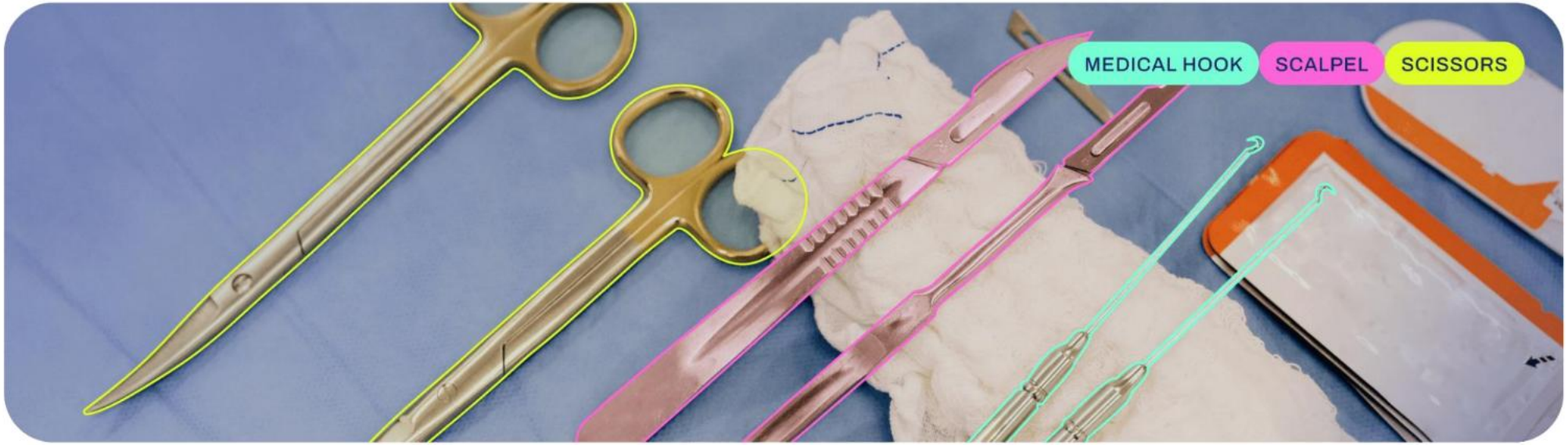
Example URL

```
from ultralytics import YOLO

# Load a pretrained YOLOv8n model
model = YOLO('yolov8n.pt')

# Define source as YouTube video URL
source = 'https://youtu.be/LNwODJXcvt4'

# Run inference on the source
results = model(source, stream=True)  # generator of Results objects
```

Parameters

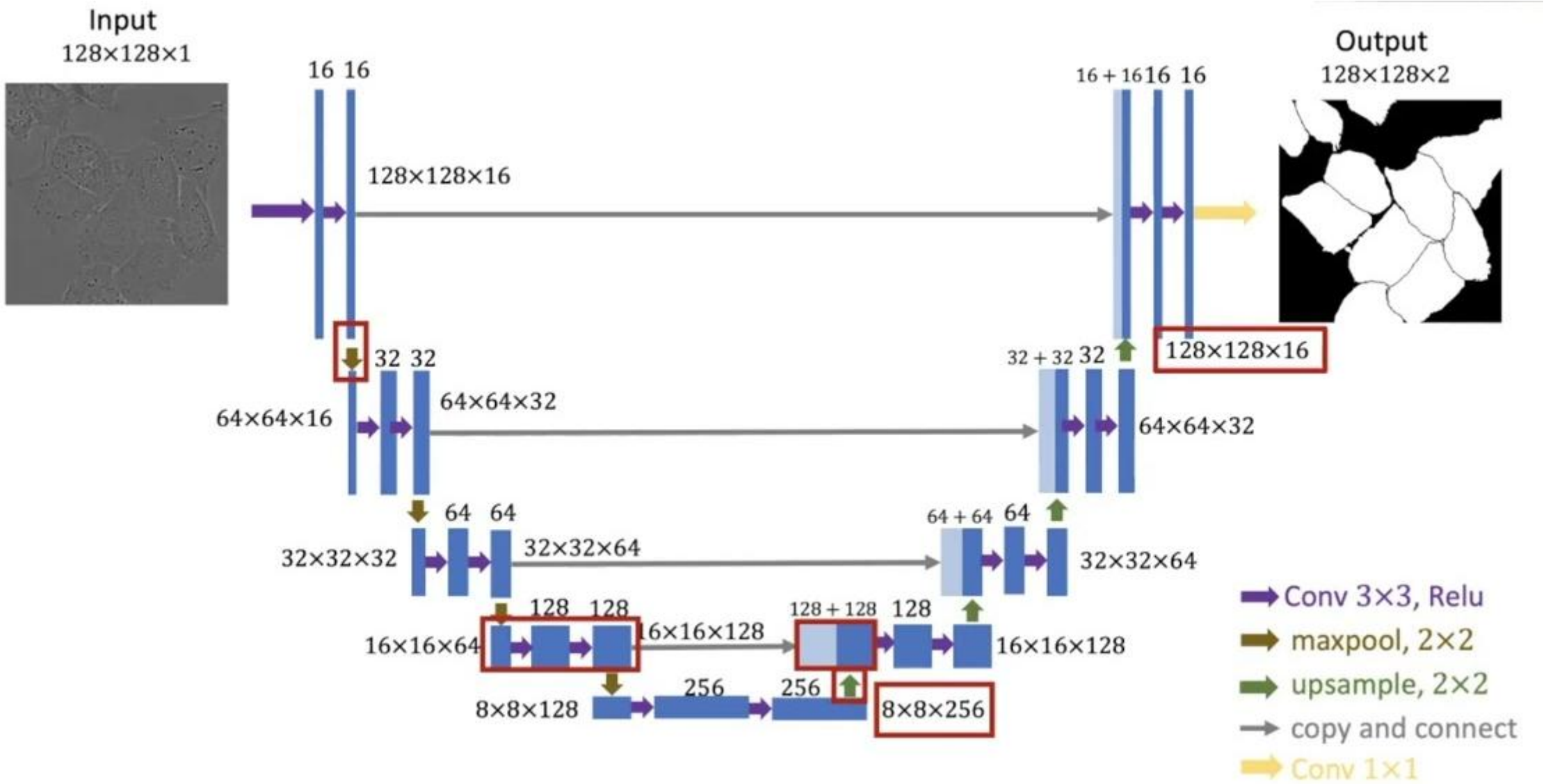
conf	float	0.25	object confidence threshold for detection
classes	list[int]	None	filter results by class, i.e. classes=0, or classes=[0,2,3]
show	bool	False	show predicted images and videos if environment allows
save	bool	False	save predicted images and videos
save_txt	bool	False	save results as `.txt` file
save_conf	bool	False	save results with confidence scores

Segmentation

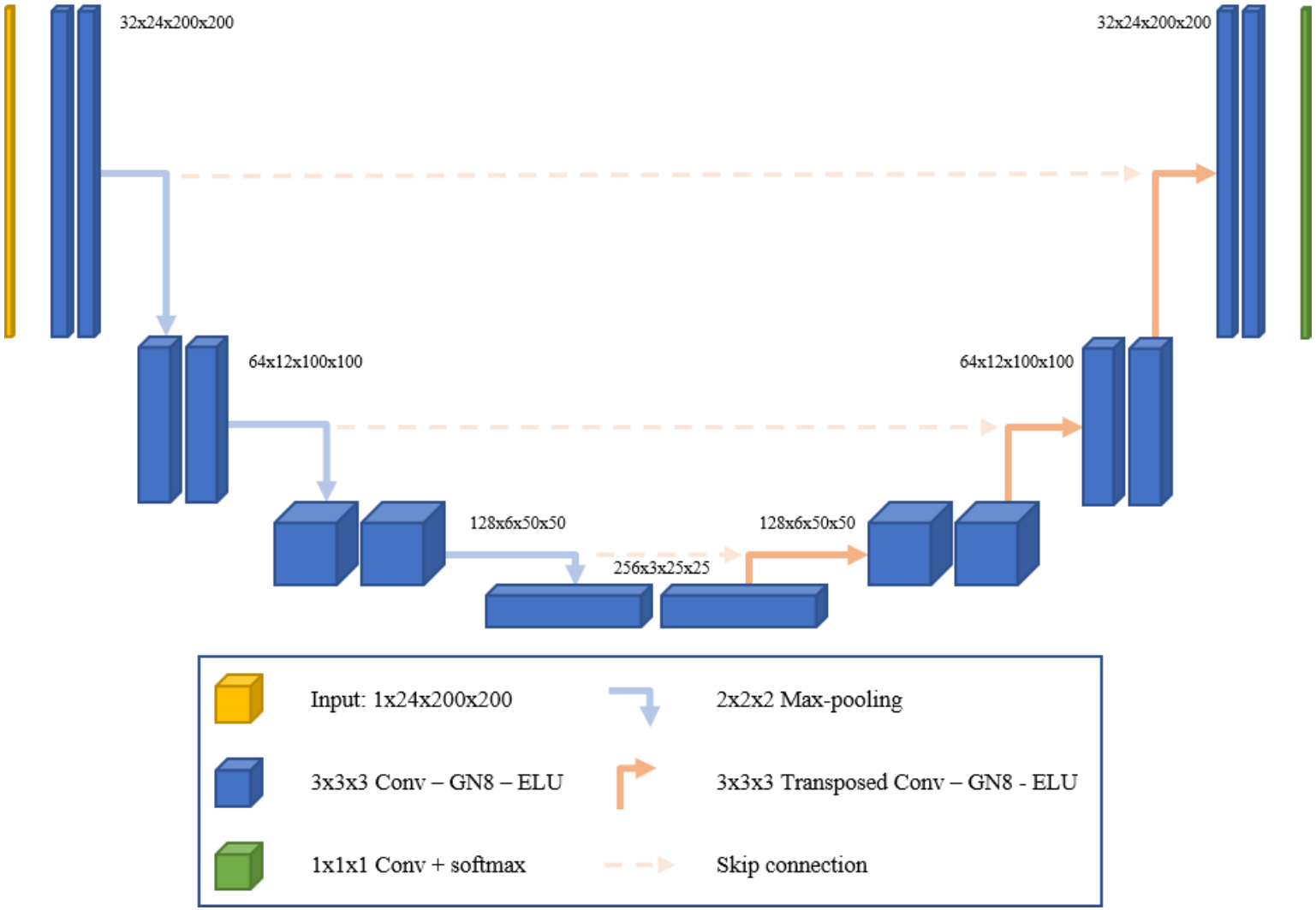


Segmentation

UNET


Input
128×128×1
16 16
128×128×16
32 32
64×64×16
64×64×32
64 64
32×32×32
32×32×64
128 128
16×16×64
16×16×128
256
8×8×128
256
8×8×256
128 + 128
128
16×16×128
64 + 64
64
32×32×64
32 + 32
32
64×64×32
16 + 16
16 16
128×128×16
Output
128×128×2
Conv 3×3, Relu
maxpool, 2×2
upsample, 2×2
copy and connect
Conv 1×1

UNET

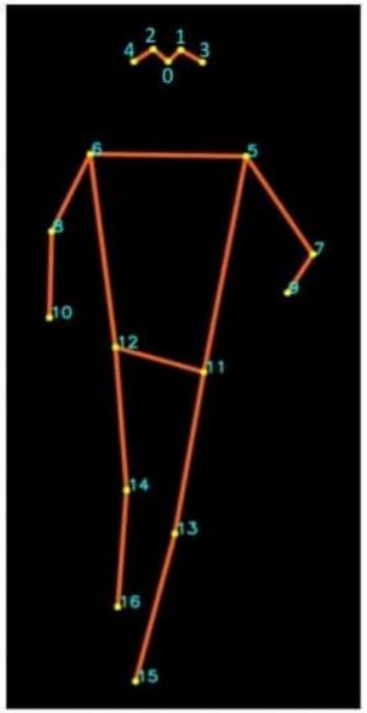

32x24x200x200
32x24x200x200
64x12x100x100
64x12x100x100
128x6x50x50
128x6x50x50
256x3x25x25
Input: 1x24x200x200
2x2x2 Max-pooling
3x3x3 Conv – GN8 – ELU
3x3x3 Transposed Conv – GN8 - ELU
1x1x1 Conv + softmax
Skip connection

Classification

```
model = YOLO('yolov8n-seg.pt')
results = model('italy.jpg', show=True)
```

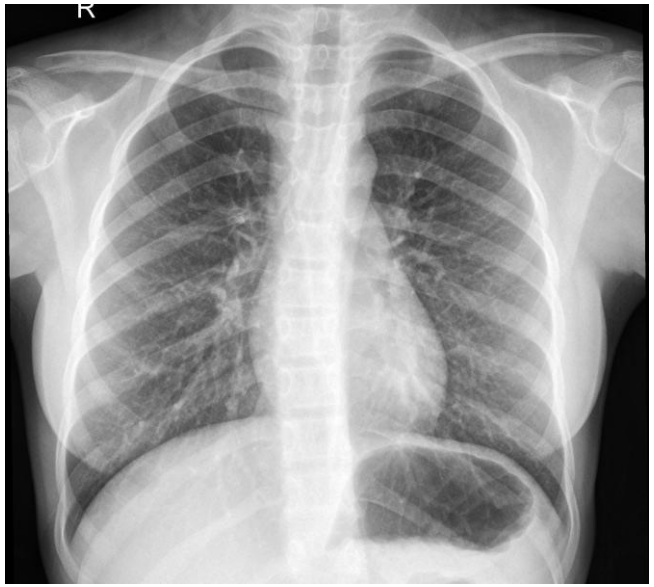
Pose Estimation

Index	Key point
0	Nose
1	Left-eye
2	Right-eye
3	Left-ear
4	Right-ear
5	Left-shoulder
6	Right-shoulder
7	Left-elbow
8	Right-elbow
9	Left-wrist
10	Right-wrist
11	Left-hip
12	Right-hip
13	Left-knee
14	Right-knee
15	Left-ankle
16	Right-ankle

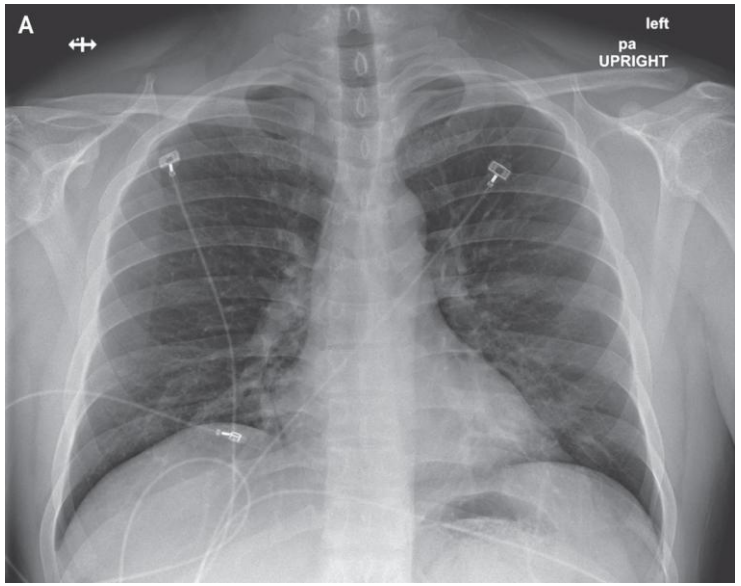


Tracking

Covid Diagnosis



Normal



Covid